

LLM Notes

2026 Spring

Day 8: Byte-pair Encoding

deeplearning

主讲: Aaron

Fudan University

今日摘要

2026.7.10 实际上的学习时间在 8.9, 中间欠了太多的日志, 慢慢补上吧..... 这两天主要再重新启动 cs336 的学习, 在 codex 的指导下, 全程手敲代码, 还是很辛苦的, 今天就不对理论做过多讲解, 转而说说具体代码实现, 以及一些高层次的领悟吧。

8.1 Tokenizer 的分类

Tokenizer 的分类并不是特别清晰。这是因为“字节级”“字符级”“子词级”等名称, 实际上可能在回答两个不同的问题:

1. Tokenizer 最底层以什么作为不可再分的原子?
2. Tokenizer 训练完成后, 输出 token 的典型粒度有多大?

定义

定义 8.1.1 (Tokenizer 的两个分类维度). Tokenizer 可以从两个相对独立的维度理解:

- **底层表示**: 最基础、不可继续拆分的符号是什么;
- **输出粒度**: 训练后的一个 token 通常对应字节、字符、子词还是完整单词。

8.1.1 底层原子

常见的底层原子包括 byte、Unicode code point, 以及某种意义上的 character。

Byte 一共只有 256 种可能取值。以 UTF-8 为例, 一个 Unicode 字符会先被编码成长度可变的 byte 序列。因此, 有限的 byte 原子通过变长组合, 便可以表示规模极大的 Unicode 字符集合。

Unicode code point 则直接为字符分配整数编号。它的符号空间远大于 256，若直接将所有码点都作为基础 token，初始词表会非常大，而且大量低频字符对应的 token 很难得到充分训练。

备注

“Character-level”本身也是一个略显含糊的说法。在许多实现中，所谓 character 实际上接近 Unicode code point；但从语言学或用户视觉角度看，一个可见字符也可能由多个 code point 组合而成。因此，character 并不总是一个严格、唯一的底层单位。

8.1.2 输出 token 的粒度

从输出粒度来看，Tokenizer 可以被描述为字节级、字符级、子词级或词级。不过，这些粒度并不一定具有绝对清晰的边界。

以 byte-level BPE 为例，它最初以单个 byte 作为基础原子，但经过不断合并后，最终 token 可能对应：

- 单个 byte；
- 一个完整字符的 UTF-8 byte 序列；
- 一个常见子词；
- 一个完整单词；
- 空格、标点与单词组成的常见片段。

分析

Tokenizer 的“字节级”和“子词级”并不是互斥分类。

“字节级”描述的是底层表示：文本先被编码为 UTF-8 bytes，并以 256 种 byte 作为不会缺失的基础词表。

“子词级”描述的是训练后的典型 token 粒度：BPE 将高频的相邻 token 反复合并，最终形成长度可变的 byte 序列。因此，GPT-2 风格的 Tokenizer 更准确地说，是一种 **byte-level BPE subword tokenizer**。

8.2 BPE 训练的大致实现

BPE 训练需要对文本进行几个层次的处理。首先将文本划分为较粗的片段，再将其转换成最基础的 byte token，最后根据相邻 pair 的频率反复进行合并。

训练的最终输出包括：

- **vocab**：从 token ID 到 token bytes 的映射；
- **merges**：按照学习顺序记录的 byte-token pair 合并规则。

8.2.1 文本的三层处理

第一层是根据 special token 切分文本。Special token 是人为加入训练语料的特殊标记，例如 `<endoftext>`。它需要作为完整 token 加入 vocab，同时充当文本边界，不参与普通的 BPE merge。

第二层使用特定的正则表达式进行 pre-tokenization，从文本中提取一个个 pre-token。这样可以限制 BPE 的合并范围，避免 token 跨越 pre-token 边界。

例如，在 “I am Aaron” 中，如果没有 pre-token 边界，理论上可能出现将单词末尾与下一个单词开头进行合并的情况。Pre-tokenization 可以阻止这种跨边界合并。

备注

Pre-tokenization 并不真正理解“语义”，它只是通过人为设计的正则规则规定允许合并的范围。因此，pre-token 更接近一种工程上的粗粒度文本片段，而不一定是严格的语言学单词。

第三层是将每个 pre-token 编码成 UTF-8 bytes，并进一步拆分为单 byte token。BPE 随后在这些基础 token 上反复执行合并，逐渐得到长度可变的子词 token。

整体流程可以概括为：

text $\longrightarrow$ special-token segments $\longrightarrow$ pre-tokens $\longrightarrow$ single-byte tokens $\longrightarrow$ merged tokens.

8.2.2 Pair 计数与合并

对于每种当前 token 序列，需要统计其中所有相邻 pair 的出现次数。若某个 pre-token 序列 s 在语料中出现了 $f(s)$ 次，那么 pair (a, b) 的全局频率可以写成：

$$C(a, b) = \sum_s f(s) \text{occ}_s(a, b),$$

其中， $\text{occ}_s(a, b)$ 表示 pair (a, b) 在序列 s 中出现的次数。

每轮训练选择频率最高的 pair，将它合并成一个新的 token，并相应更新 vocab、merges、当前 token 序列以及 pair 频率。

8.2.3 当前实现与优化思路

我目前实现的是一个正确性优先的 baseline：每完成一轮合并，就遍历更新后的 pre-token 序列，重新统计所有 pair 的频率。

这种方法比较直观，也更容易保证正确，但需要在每轮 merge 后重新扫描当前状态，因此没有通过作业中的速度测试。

更高效的版本可以维护从 pair 到相关 pre-token 的倒排索引。选出 best pair 后，只处理真正包含该 pair 的 pre-token，并对受影响的局部 pair 做增量更新。这种实现速度更快，但需要维护更多相互关联的状态，我目前还没有实现。

分析

一个已经实现的小优化，是先统计不同 pre-token 的出现次数。

例如：

```
am am am am cat cat cat cat
```

其中相同的 pre-token 会反复出现。没有必要对每次出现都重新编码和拆分；只需为每种不同的 pre-token 保存一份当前 token 序列，并记录其频率。统计 pair 时，再将该序列内部的 pair 次数乘以 pre-token 的出现频率即可。这种方法将重复计算转换成了加权计数。

备注

需要区分 BPE 的“训练”和“编码”：

- 训练阶段根据语料中的 pair 频率学习 merges；
- 编码阶段不再统计频率，而是按照已经学习好的 merge 顺序，将新文本转换为 token IDs。

8.2.4 BPE 训练与编码的相似性

在实现 Tokenizer 的过程中，我发现 BPE 训练与编码包含很多相似操作。二者都需要先处理 special token，再进行 pre-tokenization，并保证后续 merge 不会跨越这些边界。

在单个 pre-token 内，二者也都需要维护当前 token 序列、扫描相邻 pair，并通过从左到右的方式完成非重叠合并。因此，它们共享的核心操作可以概括为：

当前 token 序列 + 选定的 pair → 合并后的 token 序列。

例如，对于序列

$$(a, a, a)$$

合并 pair (a, a) 后，结果应为

$$(aa, a),$$

而不是发生重叠合并，得到两个 aa 。

分析

BPE 训练与编码的主要区别，不在于“如何执行一次合并”，而在于“如何选择下次要合并的 pair”。

训练阶段尚未得到 merge 规则，因此需要根据整个训练语料中的频率进行选择：

$$(a^*, b^*) = \arg \max_{(a,b)} C(a, b).$$

编码阶段已经拥有按训练顺序排列的 merges，因此不再统计语料频率，而是从当前可用的 pair 中选择 merge rank 最小者：

$$(a^*, b^*) = \arg \min_{(a,b) \in P_{\text{available}}} \text{rank}(a, b).$$

因此，训练是在**学习规则**，编码是在**重放规则**。

训练与编码的共同部分包括：

- 都不能跨 special token 边界进行合并；
- 都不能跨 pre-token 边界进行合并；
- 都以 byte token 序列作为初始状态；
- 都需要检查当前序列中的相邻 pair；
- 都使用从左到右的非重叠合并；
- 每次合并后，都需要基于新的 token 序列继续处理。

它们的不同之处可以概括如下：

	BPE 训练	Tokenizer 编码
目标	学习 vocab 和 merges	将新文本转换为 token IDs
选择依据	pair 的全局频率	已学习的 merge rank
处理范围	整个训练语料	当前输入中的各个 pre-token
状态更新	更新频率、vocab 与 merges	更新当前 token 序列
最终输出	vocab 和 merges	token ID 序列